

Brainbase Router Product Architecture and Protocol Design

Product Goal

Brainbase Router is a production-grade routing layer that behaves like a single LLM from the outside, but internally chooses the best model, workflow, cascade, or routing strategy for each request.

The customer sees:

```
model="brainbase-router-v1"
```

Internally, Brainbase may route to:

```
Claude
GPT
Gemini
Qwen
DeepSeek
Kimi
GLM
fine-tuned Brainbase models
customer-specific models
self-hosted models
multi-model workflows
```

The core product idea is:

```
Stable product shell
+
Replaceable router brain
+
Replaceable model registry
+
Portkey execution gateway
+
Kafka/Postgres data and training loop
```

The router brain should be modular, trainable, replaceable, and versioned like a model artifact.

Part 1: Extremely High-Level Architecture

One-line architecture

```
Customer / Brainbase Agent Platform
    ↓
OpenAI-compatible Brainbase Router API
    ↓
Brainbase Router Runtime
    ↓
Swappable RouterPolicy plugin
    ↓
RoutePlan
    ↓
Portkey Executor
    ↓
OpenRouter / Direct Providers / Self-hosted Models
    ↓
Response + Logs + Training Data
```

Core idea

The product should not be hardcoded around HyDRA, GraphRouter, LLMRouter, Avengers-Pro, RouteNLP, or any single router paper.

Instead, Brainbase should build a stable **Router Runtime** where any router can be plugged in if it follows the internal Brainbase Router Protocol.

```
Brainbase Router Product
  ≠ HyDRA product
  ≠ GraphRouter product
  ≠ LLMRouter wrapper

Brainbase Router Product
  = router operating system
```

Mental model

```
Product shell = stable puzzle
RouterPolicy = replaceable puzzle piece
RoutePlan = language every puzzle piece must speak
Portkey = execution layer
```

```
Model registry = available model inventory
Kafka/Postgres = memory and learning loop
Eval harness = promotion gate
```

What customers experience

Customers should use Brainbase Router like any normal OpenAI-compatible model.

```
from openai import OpenAI

client = OpenAI(
    api_key="BRAINBASE_API_KEY",
    base_url="https://api.brainbase.ai/v1"
)

response = client.chat.completions.create(
    model="brainbase-router-v1",
    messages=[
        {"role": "user", "content": "Debug this Kafka consumer issue."}
    ]
)

print(response.choices[0].message.content)
```

The customer does not need to know whether Brainbase routed the request to Claude, GPT, Gemini, Qwen, a cascade, or a Brainbase fine-tuned model.

What Brainbase controls internally

Brainbase controls:

```
which router policy is live
which model pool is available
which tenants can use which models
which policies are shadow-tested
which models are calibrated
which router versions are promoted
which feedback becomes training data
```

The core proprietary asset is not just “calling many models.”

The core proprietary asset is:

```
routing data
routing protocol
router training loop
agent outcome feedback
model performance profiles
customer-specific routing behavior
cost/latency/quality optimization
```

Part 2: High-Level Production Architecture

Recommended production stack

```
External API:
  OpenAI-compatible API

Router Runtime:
  Brainbase Router Runtime Kernel

Router Policies:
  HyDRA-style policy
  GraphRouter policy
  Avengers-Pro-style policy
  RouteLLM / MFRouter baseline
  Brainbase-trained router
  customer-specific router

Execution Gateway:
  Portkey

Model Marketplace:
  OpenRouter first

Direct Providers:
  Anthropic
  OpenAI
  Google/Gemini
  AWS Bedrock
  Azure OpenAI
  Together/Fireworks/etc.

Self-hosted Later:
  vLLM
  SGLang
```

Ollama
fine-tuned open-source models

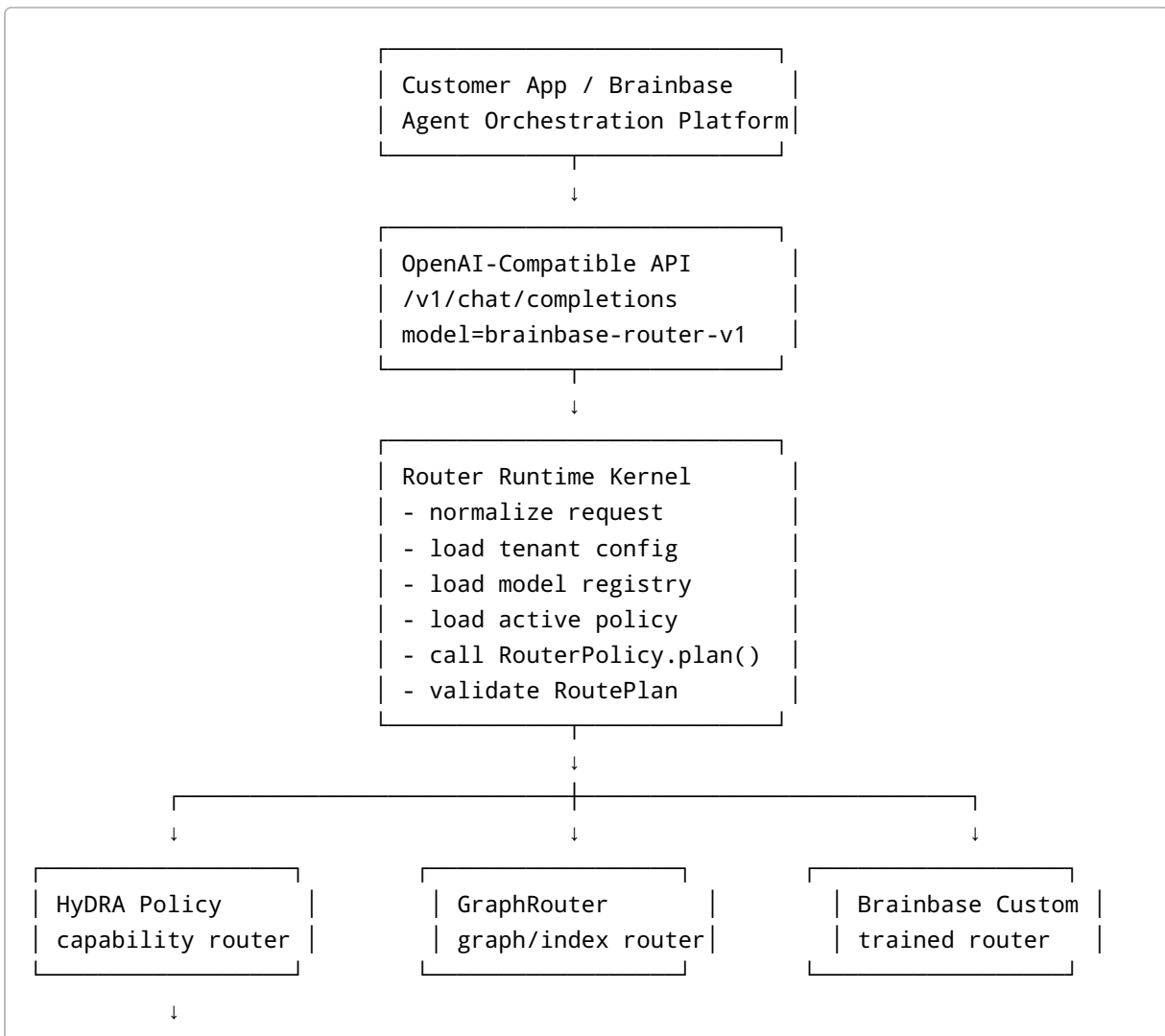
Data Layer:

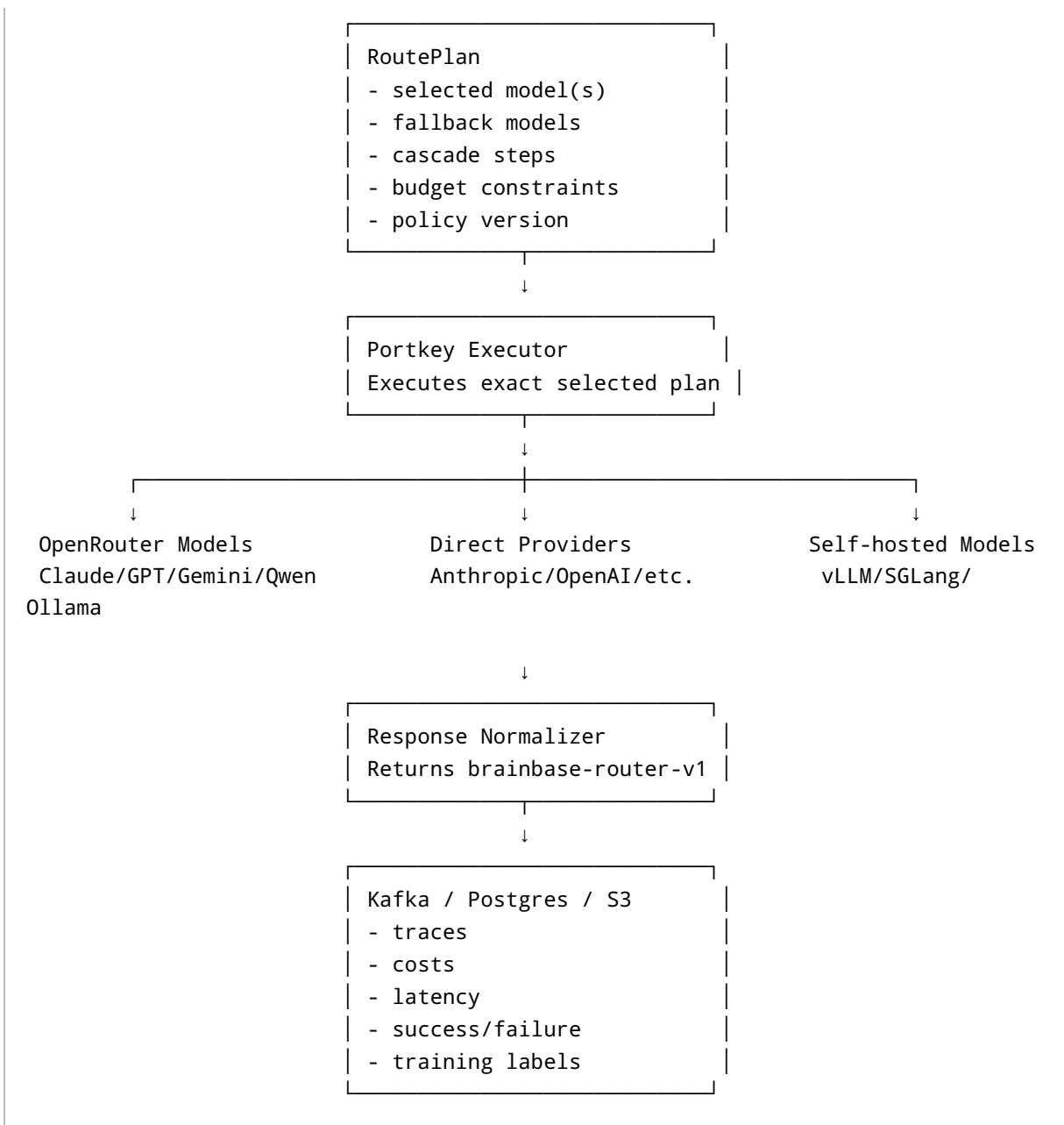
Postgres
S3/blob storage
Kafka

Evaluation:

public benchmarks
internal Brainbase replay evals
shadow testing
A/B tests
customer outcome metrics

Full architecture diagram





Main separation of responsibilities

Brainbase Router Runtime

Responsible for:

```
request normalization
tenant policy selection
model candidate filtering
router policy loading
```

```
RoutePlan validation
fallback safety
shadow routing
logging
training data export
A/B testing
```

RouterPolicy plugin

Responsible for:

```
deciding what should happen for this request
choosing model(s)
choosing cascade steps
choosing token budgets
choosing fallback strategy
producing a valid RoutePlan
```

Portkey

Responsible for:

```
executing selected model
managing provider credentials
observability
gateway-level retries if explicitly allowed
provider abstraction
```

Portkey should not secretly become the router brain during benchmark mode.

For clean benchmarking:

```
disable Portkey cache
disable hidden Portkey routing
disable hidden Portkey fallbacks
log every retry/fallback clearly
force exact selected model execution
```

Model Registry

Responsible for:

```
listing available models
tracking model capabilities
tracking costs
tracking latency
tracking tool/vision/json support
tracking tenant access
tracking model status
```

Kafka/Postgres/S3

Responsible for:

```
request trace storage
routing decision storage
model output storage
cost/latency logging
agent outcome tracking
human feedback
training/eval dataset creation
```

Part 3: Detailed Architecture and Protocol

Why the protocol matters

If the product directly depends on HyDRA, GraphRouter, or any one router library, every new paper or internal improvement creates product churn.

The better design is:

```
Product depends only on Brainbase Router Protocol.
Router algorithms adapt into that protocol.
```

This is similar to how ML inference can swap model weights while keeping the serving stack stable.

For Brainbase:

```
Old world:
change model weights → new inference behavior

Brainbase router world:
```

```
change RouterPolicy artifact → new routing behavior  
change model registry → new model availability
```

Core protocol

The protocol should be:

```
RouterPolicy.plan(...) → RoutePlan
```

Not:

```
RouterPolicy.select(...) → selected_model
```

Why?

Because modern routers may do more than pick one model.

Some may:

```
pick one model  
pick a cascade  
pick multiple models and aggregate  
route each agent step  
choose a token budget  
choose context/memory  
update online from feedback  
abstain and escalate
```

A simple `selected_model` output is too narrow.

A `RoutePlan` output is future-proof.

Core objects

RouterRequest

```
@dataclass  
class RouterRequest:  
    messages: list[dict]  
    tools: list[dict] | None
```

```
response_format: dict | None
modality: str
tenant_id: str
workflow_id: str | None
step_id: str | None
latency_budget_ms: int | None
cost_budget_usd: float | None
quality_mode: str
metadata: dict
```

Example:

```
{
  "messages": [
    {"role": "user", "content": "Fix this Python async bug."}
  ],
  "tools": null,
  "modality": "text",
  "tenant_id": "acme",
  "workflow_id": "wf_123",
  "step_id": "debug_step_2",
  "latency_budget_ms": 3000,
  "cost_budget_usd": 0.02,
  "quality_mode": "balanced"
}
```

ModelProfile

```
@dataclass
class ModelProfile:
    id: str
    executor_model: str
    provider: str
    capabilities: dict
    cost: dict
    latency: dict
    limits: dict
    supports: dict
    status: str
    tenant_access: list[str]
```

Example:

```
id: claude-sonnet-4.6
executor_model: "@openrouter/anthropic/claude-sonnet-4.6"
provider: openrouter
status: enabled
supports:
  tools: true
  vision: true
  json: true
  streaming: true
limits:
  context_window: 200000
capabilities:
  reasoning: 0.88
  coding: 0.91
  debugging: 0.87
  tool_use: 0.90
  latency_score: 0.62
cost:
  input_per_million: 3.0
  output_per_million: 15.0
```

RoutingContext

```
@dataclass
class RoutingContext:
    tenant_config: dict
    workflow_context: dict | None
    conversation_summary: str | None
    historical_performance: dict | None
    policy_config: dict
```

This allows routing to be tenant-aware and workflow-aware.

Example use cases:

```
customer A prefers lower cost
customer B prefers best quality
voice workflows prefer low latency
legal workflows require higher reliability
tool-generation steps prefer strong function calling
```

RoutingBudget

```
@dataclass
class RoutingBudget:
    max_cost_usd: float | None
    max_latency_ms: int | None
    max_cascade_depth: int
    allow_multi_call: bool
    allow_expensive_models: bool
    quality_mode: str
```

RoutePlan

```
@dataclass
class RoutePlan:
    mode: str
    selected_model: str | None
    steps: list[RouteStep]
    fallback_models: list[str]
    confidence: float
    policy_name: str
    policy_version: str
    metadata: dict
```

RouteStep

```
@dataclass
class RouteStep:
    model: str
    role: str | None
    condition: str | None
    max_tokens: int | None
    timeout_ms: int | None
    verifier: str | None
    metadata: dict
```

RoutePlan modes

The protocol should support typed modes.

1. Single-model route

Used by:

```
HyDRA  
GraphRouter  
RouteLLM  
MFRouter  
simple Brainbase heuristic router
```

Example:

```
{  
  "mode": "single",  
  "selected_model": "claude-sonnet-4.6",  
  "fallback_models": ["gpt-5.4", "claude-opus-latest"],  
  "confidence": 0.87,  
  "policy_name": "hydra",  
  "policy_version": "v2"  
}
```

2. Cascade route

Used by:

```
FrugalGPT  
AutoMix  
RouteNLP  
conformal cascade routers
```

Example:

```
{  
  "mode": "cascade",  
  "steps": [  
    {  
      "model": "qwen-coder-fast",  
      "condition": "try_first"  
    },  
    {  
      "model": "claude-sonnet-4.6",  
      "condition": "try_second"  
    }  
  ]  
}
```

```
    "condition": "if_confidence_below_0.75"
  },
  {
    "model": "claude-opus-latest",
    "condition": "if_verifier_fails"
  }
],
"policy_name": "routenlp",
"policy_version": "v1"
}
```

3. Multi-call route

Used by:

```
Router-R1-style systems
planner-executor-judge systems
multi-model aggregation
```

Example:

```
{
  "mode": "multi_call",
  "steps": [
    {
      "model": "gemini-pro",
      "role": "planner"
    },
    {
      "model": "qwen-coder",
      "role": "implementation"
    },
    {
      "model": "claude-sonnet",
      "role": "judge_merge"
    }
  ],
  "policy_name": "router-r1",
  "policy_version": "v1"
}
```

4. Agent-step route

Used by:

```
Brainbase Kafka workflows  
TwinRouterBench-style routing  
MTRouter  
multi-turn routers  
agentic workflows
```

Example:

```
{  
  "mode": "agent_step",  
  "selected_model": "qwen-coder",  
  "metadata": {  
    "workflow_step": "tool_argument_generation",  
    "history_window": "last_8_steps"  
  }  
}
```

5. Budgeted generation route

Used by:

```
R2-Router-style systems  
token-budget-aware routers  
latency/cost-sensitive products
```

Example:

```
{  
  "mode": "budgeted_single",  
  "selected_model": "gpt-5.4",  
  "generation": {  
    "max_tokens": 700,  
    "length_instruction": "Answer concisely."  
  }  
}
```

6. Context-routing route

Used by:

```
memory routers
long-context routers
RAG/context selectors
agent memory routing
```

Example:

```
{
  "mode": "context_routing",
  "selected_model": "claude-sonnet",
  "context_plan": {
    "include_memory_ids": ["m1", "m9", "m22"],
    "max_context_tokens": 12000
  }
}
```

RouterPolicy interface

```
class RouterPolicy:
    name: str
    version: str
    supported_modes: list[str]

    def plan(
        self,
        request: RouterRequest,
        candidates: list[ModelProfile],
        context: RoutingContext,
        budget: RoutingBudget
    ) -> RoutePlan:
        ...
```

Optional interfaces

Trainable policy

```
class TrainableRouterPolicy(RouterPolicy):
    def train(
        self,
        dataset_uri: str,
        output_artifact_uri: str,
        config: dict
    ) -> RouterArtifact:
        ...
```

Online learning policy

```
class OnlineLearningRouterPolicy(RouterPolicy):
    def update(
        self,
        feedback: RouterFeedbackEvent
    ) -> None:
        ...
```

Explainable policy

```
class ExplainableRouterPolicy(RouterPolicy):
    def explain(
        self,
        request: RouterRequest,
        route_plan: RoutePlan
    ) -> dict:
        ...
```

Part 4: Full Technical Analysis, Router Types, Pros, Cons, and Mitigations

Why this architecture works

This architecture separates:

```
product shell  
router algorithm  
model execution  
model inventory  
feedback loop  
training lifecycle
```

That means a change in one area does not force a rewrite everywhere else.

What happens when things change?

New router algorithm comes out

```
implement RouterPolicy adapter  
package artifact  
register policy  
run offline eval  
run shadow eval  
promote if better
```

New trained router is ready

```
upload checkpoint/artifact  
update policy version  
shadow test  
promote
```

New foundation model comes out

```
add model profile  
calibrate capabilities  
enable in model registry  
router can select it
```

New fine-tuned LLM is ready

```
register as model  
add capability scores  
enable for tenants  
router can select it
```

New customer-specific behavior is needed

```
create tenant policy config
assign model pool
assign routing mode
run customer-specific evals
```

Router category analysis

A. Single-model routers

Examples:

```
GraphRouter
RouteLLM
MFRouter
basic heuristic router
```

How they work

They receive a request and choose one model.

```
request → router → selected_model
```

Protocol mode

```
mode="single"
```

Pros

```
simple
fast
cheap
easy to benchmark
easy to debug
low operational complexity
```

Cons

```
cannot express cascades
cannot express multi-model aggregation
cannot express agent-step routing well
cannot retry based on verification
```

Mitigation

Support single-model routing as one RoutePlan mode, but do not make it the entire protocol.

B. Capability-profile routers

Example:

```
HyDRA
```

How they work

They predict what capabilities the request needs, such as:

```
reasoning
code generation
debugging
tool use
latency sensitivity
```

Then they match those needs against configured model profiles and choose the cheapest model that satisfies the request.

Protocol mode

Usually:

```
mode="single"
```

Sometimes:

```
mode="cascade"
```

Pros

very product-friendly
model additions can be config-only
easy to understand
good for heterogeneous model pools
fits new-model adoption well

Cons

capability scores can become stale
manual capability scores can be wrong
hard to represent subtle model behavior
may underperform if capability schema is too small

Mitigation

Use measured calibration, not guesses.

Every model should go through:

registered
calibrating
shadow
limited
enabled

Capability profiles should be updated from:

public benchmarks
Brainbase internal evals
tenant-specific evals
live outcome data

C. Semantic clustering routers

Examples:

```
Avengers-Pro-style router
nearest-neighbor router
embedding cluster router
```

How they work

They embed the prompt, find similar historical or benchmark prompts, and select the model that performed best for similar prompts.

Protocol mode

Usually:

```
mode="single"
```

Pros

```
easy to improve with Brainbase traffic
good for domain-specific routing
good for customer-specific routing
can learn from real outcomes
```

Cons

```
can overfit to historical traffic
bad for out-of-distribution queries
depends heavily on embedding quality
needs enough labeled examples
```

Mitigation

Add OOD detection.

```
if nearest-cluster confidence is low:
    fall back to HyDRA-style capability router
    or strongest model
```

Also combine with:

```
task classifier
difficulty classifier
tenant-specific performance stats
```

D. Cascade routers

Examples:

```
FrugalGPT
AutoMix
RouteNLP
conformal cascade routers
```

How they work

They try a cheaper or faster model first, then escalate if confidence is low or verification fails.

```
cheap model
  ↓
if good enough → return
  ↓
if not good enough → stronger model
  ↓
if still not good → strongest model
```

Protocol mode

```
mode="cascade"
```

Pros

```
strong cost savings
practical for production
keeps quality high by escalating
good for mixed easy/hard traffic
```

Cons

```
latency can increase
multiple model calls per request
harder attribution
more complex logging
risk of verifier mistakes
```

Mitigation

Add:

```
max_cascade_depth
latency_budget_ms
cost_budget_usd
explicit verifier rules
step-level logs
fallback reason codes
```

E. Agent-step routers

Examples:

```
Brainbase Kafka workflows
TwinRouterBench-style routing
MTRouter
multi-turn routing
```

How they work

They route each individual model call inside a longer workflow.

A single customer request may create many internal steps:

```
classify intent
retrieve data
generate tool call
validate result
write summary
```

```
decide next action  
respond to user
```

Each step may need a different model.

Protocol mode

```
mode="agent_step"
```

Pros

```
very relevant to Brainbase  
huge cost-saving opportunity  
better model choice per workflow step  
can optimize whole agent outcome
```

Cons

```
credit assignment is hard  
workflow success may depend on many calls  
debugging becomes harder  
requires very good tracing
```

Mitigation

Every call must log:

```
workflow_id  
step_id  
parent_request_id  
selected_model  
router_policy  
policy_version  
cost  
latency  
tool success  
step success  
final workflow outcome
```

Without this, training and debugging will be weak.

F. Budgeted-output routers

Examples:

```
R2-Router-style systems  
token-budget-aware routers
```

How they work

They choose both:

```
model  
+  
generation length/token budget
```

Protocol mode

```
mode="budgeted_single"
```

Pros

```
major cost control  
better latency control  
useful for repetitive production workloads
```

Cons

```
short outputs may hurt quality  
hard to compare fairly  
models behave differently under token limits
```

Mitigation

Make output budget explicit in RoutePlan.

Do not hide it in prompts only.

Track:

```
max_tokens
actual_output_tokens
quality score
retry rate
user satisfaction
```

G. Online-learning routers

Examples:

```
OrcaRouter
BaRP
bandit routers
adaptive routing policies
```

How they work

They update from production feedback.

```
route request
observe result
update policy
improve future routing
```

Protocol requirement

Add optional:

```
update(feedback_event)
```

Pros

```
adapts to real customer traffic
can personalize by tenant
can improve after deployment
```

Cons

```
feedback is noisy
feedback is delayed
you only observe chosen model performance
bad updates can degrade production
harder to reproduce behavior
```

Mitigation

Use staged rollout:

```
offline replay
shadow mode
1% traffic
tenant-limited beta
full rollout
```

Use frozen checkpoints:

```
adaptive-router-v3.1
adaptive-router-v3.2
adaptive-router-v3.3
```

Always allow rollback.

H. Multi-call and aggregation routers

Examples:

```
Router-R1
planner-executor-judge workflows
multi-model judge/merge systems
```

How they work

They may call multiple models and combine outputs.

Example:

```
Gemini plans
Qwen writes code
Claude judges/merges
```

Protocol mode

```
mode="multi_call"
```

Pros

```
can outperform single models on hard tasks
good for best-quality mode
good for complex agent workflows
```

Cons

```
expensive
slow
harder to debug
judge model can be wrong
risk of over-escalation
```

Mitigation

Only allow when:

```
quality_mode="best"
or task_difficulty is high
or tenant explicitly allows it
```

Add hard:

```
cost caps
latency caps
max model calls
judge confidence threshold
```

I. Context-routing routers

Examples:

```
memory selection routers  
RAG context routers  
agent memory routers
```

How they work

They decide what context or memory should be included before calling the model.

Protocol mode

```
mode="context_routing"
```

Pros

```
important for long-context agents  
reduces token cost  
improves relevance  
helps with memory-heavy workflows
```

Cons

```
dropping important context can destroy quality  
hard to know which memory mattered  
hard to evaluate
```

Mitigation

Log:

```
included memory IDs  
dropped memory IDs  
retrieval scores  
final answer quality  
workflow success
```

J. Routers requiring hidden model internals

Some router methods may require:

```
logits
hidden states
prefill activations
model internals
```

Pros

```
can be powerful for open-source/self-hosted models
```

Cons

```
not compatible with closed APIs like Claude/GPT/Gemini
not compatible with Portkey/OpenRouter-only execution
harder to productionize
```

Mitigation

Classify routers by compatibility level.

```
Level 1:
  API-only compatible
  Works with Claude/GPT/Gemini/OpenRouter

Level 2:
  Needs logprobs or extra outputs
  Works only if provider exposes them

Level 3:
  Needs internals
  Works only with self-hosted/open-source models
```

For general Brainbase production, prioritize Level 1.

Part 5: Protocol Pros, Cons, and How to Deal With Them

Protocol advantage 1: Router algorithms become swappable

Benefit

New router papers become plugins.

```
HyDRA → HyDRAPolicy  
GraphRouter → GraphRouterPolicy  
Avengers-Pro → AvengersPolicy  
Brainbase-trained router → BrainbasePolicy
```

Risk

Adapter work may still take time.

Mitigation

Define strict certification tests.

Every router plugin must pass:

```
schema tests  
budget tests  
tool-support tests  
fallback tests  
tenant-isolation tests  
shadow replay tests  
latency tests  
determinism tests
```

Protocol advantage 2: Product shell remains stable

Benefit

The API, billing, Portkey integration, logging, evals, and dashboard do not change when the router changes.

Risk

Protocol can become too generic or vague.

Mitigation

Use typed `RoutePlan.mode`.

Do not allow arbitrary unstructured JSON to control execution.

Protocol advantage 3: Fine-tuned routers become artifacts

Benefit

A trained router can be swapped like a model checkpoint.

```
brainbase-router-v3  
brainbase-router-v4  
customer-acme-router-v1
```

Risk

Different trained routers may require different preprocessing.

Mitigation

Each router artifact must include:

```
manifest.yaml  
weights/checkpoint/index  
preprocessing config  
model profile schema  
policy adapter version  
eval report
```

Protocol advantage 4: New models are config changes

Benefit

When a new model comes out, add it to the model registry and calibrate it.

Risk

Bad capability scores cause bad routing.

Mitigation

Use model lifecycle:

```
registered  
calibrating  
shadow  
limited  
enabled  
disabled
```

Protocol advantage 5: Supports Brainbase-specific training loop

Benefit

Brainbase can train routers on actual agent outcomes, not generic benchmark scores only.

Risk

Production feedback is noisy and biased.

Mitigation

Track multiple feedback signals:

```
user thumbs up/down  
human review  
tool success  
agent success  
call outcome  
customer-specific KPI  
latency  
cost  
fallback rate
```

Use offline replay and shadow testing before live promotion.

Protocol risk: Overengineering

Problem

Trying to support every possible future router can slow down MVP.

Solution

Build the full protocol shape, but implement only the necessary modes first.

MVP modes:

```
single
cascade
agent_step
```

Later modes:

```
multi_call
budgeted_single
context_routing
online_learning
```

This keeps the interface future-proof without building everything immediately.

Part 6: Suggested MVP Implementation

MVP product architecture

```
OpenAI-compatible API
  ↓
Router Runtime Kernel
  ↓
HyDRA-style RouterPolicy v0
  ↓
RoutePlanValidator
  ↓
Portkey Executor
  ↓
OpenRouter model pool
  ↓
```

Kafka/Postgres logs

↓

Offline eval + shadow testing

MVP router policies

Start with:

```
AlwaysStrongestPolicy
AlwaysCheapestPolicy
HyDRAPolicy
GraphRouterPolicy or MFRouterPolicy as baseline
BrainbaseHeuristicPolicy
```

Then evaluate.

MVP public models

```
brainbase-router-fast
brainbase-router-balanced
brainbase-router-best
brainbase-router-code
brainbase-router-agent
```

Example mapping:

```
public_models:
  brainbase-router-fast:
    policy: hydra-v0
    mode: latency_optimized

  brainbase-router-balanced:
    policy: hydra-v0
    mode: balanced

  brainbase-router-best:
    policy: hydra-v0
    mode: quality_optimized

  brainbase-router-code:
    policy: hydra-v0
    mode: code_optimized
```

```
brainbase-router-agent:  
  policy: brainbase-agent-router-v0  
  mode: workflow_optimized
```

MVP model pool

Start with models across categories:

```
cheap/fast:  
  Haiku-class  
  Flash-class  
  small Qwen/Llama/Mistral  
  
medium:  
  Sonnet-class  
  Gemini Flash/Pro  
  GPT mid-tier  
  Qwen coder  
  DeepSeek  
  
strong:  
  Opus-class  
  GPT strongest  
  Gemini strongest  
  
specialized:  
  coding models  
  tool-use models  
  long-context models  
  Brainbase fine-tuned models later
```

MVP data to log

Every request should produce a trace:

```
{  
  "request_id": "req_123",  
  "tenant_id": "acme",  
  "public_model": "brainbase-router-balanced",  
  "policy_name": "hydra",  
  "policy_version": "v0.1",  
  "route_plan_mode": "single",  
  "selected_model": "claude-sonnet-4.6",  
  "candidate_models": ["haiku", "sonnet", "opus", "qwen-coder"],  
}
```

```
"fallback_models": ["gpt-5.4"],  
"fallback_used": false,  
"latency_ms": 1830,  
"cost_usd": 0.0042,  
"input_tokens": 1200,  
"output_tokens": 430,  
"tool_success": null,  
"workflow_success": true,  
"user_feedback": null  
}
```

Part 7: Finished Product Documentation

This section assumes the product already exists.

7.1 Customer-facing usage

What is Brainbase Router?

Brainbase Router is an OpenAI-compatible model endpoint that automatically selects the best model or workflow for each request.

Instead of choosing between Claude, GPT, Gemini, Qwen, and other models manually, customers call one Brainbase model:

```
brainbase-router-v1
```

Brainbase handles:

```
model selection  
fallbacks  
cost optimization  
latency optimization  
tool-use optimization  
agent workflow optimization
```

7.2 Basic SDK usage

Python

```
from openai import OpenAI

client = OpenAI(
    api_key="BRAINBASE_API_KEY",
    base_url="https://api.brainbase.ai/v1"
)

response = client.chat.completions.create(
    model="brainbase-router-v1",
    messages=[
        {"role": "user", "content": "Write a concise summary of this support ticket."}
    ]
)

print(response.choices[0].message.content)
```

JavaScript / TypeScript

```
import OpenAI from "openai";

const client = new OpenAI({
  apiKey: process.env.BRAINBASE_API_KEY,
  baseURL: "https://api.brainbase.ai/v1",
});

const response = await client.chat.completions.create({
  model: "brainbase-router-v1",
  messages: [
    {
      role: "user",
      content: "Debug this Node.js websocket issue."
    }
  ],
});

console.log(response.choices[0].message.content);
```

7.3 Customer-facing router modes

Customers can choose public product models:

```
brainbase-router-fast  
brainbase-router-balanced  
brainbase-router-best  
brainbase-router-code  
brainbase-router-agent
```

Fast mode

```
response = client.chat.completions.create(  
    model="brainbase-router-fast",  
    messages=[...]  
)
```

Optimized for:

```
low latency  
low cost  
simple tasks  
chat/support  
high-volume workflows
```

Balanced mode

```
response = client.chat.completions.create(  
    model="brainbase-router-balanced",  
    messages=[...]  
)
```

Optimized for:

```
good quality  
reasonable cost  
general production use
```

Best mode

```
response = client.chat.completions.create(  
    model="brainbase-router-best",  
    messages=[...]  
)
```

Optimized for:

```
highest quality  
hard reasoning  
critical workflows  
complex coding/debugging
```

Code mode

```
response = client.chat.completions.create(  
    model="brainbase-router-code",  
    messages=[...]  
)
```

Optimized for:

```
code generation  
debugging  
software engineering tasks  
SWE workflows
```

Agent mode

```
response = client.chat.completions.create(  
    model="brainbase-router-agent",  
    messages=[...],  
    tools=[...]  
)
```

Optimized for:

```
tool calls  
workflow steps  
agent orchestration
```

multi-step execution
Kafka-based Brainbase agents

7.4 Advanced customer options

Customers may optionally pass routing preferences.

```
response = client.chat.completions.create(  
    model="brainbase-router-v1",  
    messages=[...],  
    extra_body={  
        "routing": {  
            "mode": "balanced",  
            "max_cost_usd": 0.02,  
            "latency_budget_ms": 3000,  
            "allow_multi_call": False,  
            "debug": True  
        }  
    }  
)
```

Possible routing options:

```
mode:  
    fast  
    balanced  
    best  
    cheapest  
    code  
    agent  
  
max_cost_usd:  
    maximum budget for the request  
  
latency_budget_ms:  
    maximum acceptable latency  
  
allow_multi_call:  
    whether Brainbase can call multiple models  
  
debug:  
    whether to return routing metadata
```

7.5 Debug response

For normal customers, Brainbase hides internal routing.

Normal response:

```
{
  "model": "brainbase-router-v1",
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "..."
      }
    }
  ]
}
```

For debug customers:

```
{
  "model": "brainbase-router-v1",
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "..."
      }
    }
  ],
  "brainbase_routing": {
    "policy_name": "hydra",
    "policy_version": "v0.2.1",
    "route_plan_mode": "single",
    "selected_model": "claude-sonnet-4.6",
    "fallback_used": false,
    "confidence": 0.87,
    "latency_ms": 1840,
    "cost_usd": 0.0041
  }
}
```

Part 8: Internal Brainbase Documentation

8.1 Adding a new router algorithm

Example:

New paper comes out: SuperRouter

Steps:

1. Implement SuperRouterPolicy adapter.
2. Make it implement RouterPolicy.plan().
3. Package required artifacts.
4. Register it in policy registry.
5. Run offline benchmark replay.
6. Run production shadow mode.
7. Compare with current live policy.
8. Promote if better.

Example adapter:

```
class SuperRouterPolicy(RouterPolicy):
    name = "superrouter"
    version = "v1"
    supported_modes = ["single", "cascade"]

    def plan(
        self,
        request: RouterRequest,
        candidates: list[ModelProfile],
        context: RoutingContext,
        budget: RoutingBudget
    ) -> RoutePlan:
        native_result = self.superrouter.predict(
            request=request,
            models=candidates,
            context=context
        )

        return RoutePlan(
            mode="single",
            selected_model=native_result.model_id,
            steps=[],
```

```
        fallback_models=native_result.fallbacks,  
        confidence=native_result.confidence,  
        policy_name=self.name,  
        policy_version=self.version,  
        metadata=native_result.metadata  
    )
```

Register:

```
policies:  
  - name: superrouter  
    version: v1  
    artifact_uri: s3://brainbase-router-artifacts/superrouter/v1  
    status: shadow
```

Promote:

```
active_policy:  
  name: superrouter  
  version: v1
```

8.2 Adding a new Brainbase-trained router

Steps:

1. Train router on Brainbase routing dataset.
2. Export checkpoint/artifact.
3. Create manifest.yaml.
4. Register artifact.
5. Run offline replay.
6. Run shadow mode.
7. Promote if better.

Artifact structure:

```
router_artifacts/  
  brainbase-trained-router-v4/  
    manifest.yaml  
    checkpoint.pt  
    feature_config.yaml  
    preprocessing_config.yaml
```

```
model_profile_schema.yaml
eval_report.json
```

Manifest example:

```
name: brainbase-trained-router
version: v4
type: neural_policy
supported_modes:
  - single
  - cascade
requires:
  embeddings: true
  model_profiles: true
  workflow_context: true
artifact:
  checkpoint: checkpoint.pt
  preprocessing: preprocessing_config.yaml
  eval_report: eval_report.json
```

Policy registry:

```
policies:
  - name: brainbase-trained-router
    version: v4
    artifact_uri: s3://brainbase-router-artifacts/brainbase-trained-router/v4
    status: shadow
```

8.3 Adding a new underlying LLM

Example:

```
New Claude/GPT/Gemini/Qwen model comes out
```

Steps:

1. Add model to model registry.
2. Mark status as registered.
3. Run calibration evals.
4. Update capability scores.
5. Enable in shadow mode.

6. Enable for limited tenants.
7. Enable globally if successful.

Model registry entry:

```
models:
  - id: claude-opus-new
    display_name: Claude Opus New
    executor: portkey
    executor_model: "@openrouter/anthropic/claude-opus-new"
    provider: openrouter
    status: registered
    supports:
      tools: true
      vision: true
      json: true
      streaming: true
    limits:
      context_window: 200000
    cost:
      input_per_million: null
      output_per_million: null
    capabilities:
      reasoning: null
      coding: null
      debugging: null
      tool_use: null
      latency_score: null
```

After calibration:

```
status: shadow
capabilities:
  reasoning: 0.94
  coding: 0.91
  debugging: 0.89
  tool_use: 0.90
  latency_score: 0.52
```

Then:

```
status: enabled
```

8.4 Adding a fine-tuned LLM

If Brainbase fine-tunes a model, that is not a new router policy. It is a new model in the model registry.

Example:

```
models:
- id: brainbase-qwen-agent-v1
  display_name: Brainbase Qwen Agent v1
  executor: portkey
  executor_model: "@openrouter/brainbase/qwen-agent-v1"
  provider: custom
  status: shadow
  supports:
    tools: true
    vision: false
    json: true
    streaming: true
  limits:
    context_window: 128000
  cost:
    input_per_million: 0.20
    output_per_million: 0.80
  capabilities:
    reasoning: 0.72
    coding: 0.84
    debugging: 0.81
    tool_use: 0.91
    voice_agent: 0.93
    latency_score: 0.88
```

Lifecycle:

```
fine-tune model
register model
calibrate model
shadow model
enable for selected tenants
enable globally if useful
```

8.5 Difference between fine-tuned router and fine-tuned LLM

Fine-tuned router

Update:

```
RouterPolicy artifact
```

Example:

```
active_policy:
  name: brainbase-trained-router
  version: v4
```

Fine-tuned LLM

Update:

```
Model registry
```

Example:

```
models:
  - id: brainbase-qwen-agent-v2
```

The router may then start selecting the new fine-tuned LLM.

8.6 Shadow testing

Shadow testing means the new router makes decisions but does not control production.

Flow:

```
request comes in
↓
live router chooses model and executes
↓
shadow router also creates RoutePlan
↓
shadow decision is logged
```

↓
compare live vs shadow offline

Logged comparison:

```
{
  "request_id": "req_123",
  "live_policy": "hydra-v2",
  "live_selected_model": "claude-sonnet-4.6",
  "shadow_policy": "brainbase-trained-router-v4",
  "shadow_selected_model": "qwen-coder-fast",
  "actual_outcome": "success",
  "estimated_shadow_cost": 0.0012,
  "estimated_live_cost": 0.0042
}
```

Shadow testing prevents risky router changes from immediately affecting users.

8.7 Promotion process

A router can be promoted only if it beats the current live router on defined metrics.

Metrics:

```
quality score
cost per successful task
latency p50/p95
workflow success rate
tool success rate
fallback rate
customer feedback
error rate
```

Promotion flow:

```
offline benchmark pass
↓
Brainbase replay eval pass
↓
shadow traffic pass
↓
1% live traffic
```

```
↓  
10% live traffic  
↓  
tenant-specific rollout  
↓  
global rollout
```

8.8 Rollback process

Every active router must be versioned.

Example:

```
active_policy:  
  name: brainbase-trained-router  
  version: v4  
previous_policy:  
  name: hydra  
  version: v2
```

Rollback:

```
active_policy:  
  name: hydra  
  version: v2
```

Rollback should not require code deployment.

Part 9: Benchmarking and Expected Claims

What to benchmark against

Every Brainbase router should be compared against:

```
always strongest model  
always cheapest model  
always fastest model  
HyDRA-style router  
GraphRouter / MFRouter
```

Avengers-Pro-style router
current production router
customer-specific router

Metrics

quality
cost
latency
tokens
tool-call success
workflow success
fallback rate
escalation rate
human preference
cost per successful task

The most important production metric is:

cost per successful workflow

Not just:

raw benchmark accuracy

Realistic initial expectation

The first production router should not claim:

always smarter than Opus/GPT/Gemini

The better claim is:

near-frontier quality at materially lower cost and latency

Expected MVP targets:

quality: 95-102% of strongest-model baseline depending on task mix
cost saving: 20-55% depending on aggressiveness

latency improvement: significant when many calls route to small/fast models
raw quality above strongest model: possible in domain-specific workflows, but must be measured

Best commercial/product positioning

Brainbase Router automatically selects the best model or workflow for each request, optimizing for quality, cost, latency, and agent success. It starts from strong public routing baselines and improves over time using Brainbase-specific production feedback.

Part 10: Recommended Repo Structure

```
brainbase-router/  
  api/  
    openai_compatible_server.py  
    auth.py  
    rate_limits.py  
  
  runtime/  
    kernel.py  
    request_normalizer.py  
    route_plan.py  
    route_plan_validator.py  
    policy_loader.py  
    fallback_manager.py  
    shadow_runner.py  
    ab_test_manager.py  
  
  protocol/  
    router_request.py  
    model_profile.py  
    routing_context.py  
    routing_budget.py  
    route_plan.py  
    feedback_event.py  
  
  policies/  
    base.py  
    always_strongest/  
      policy.py  
    always_cheapest/
```

```
    policy.py
hydra/
  policy.py
  artifact_loader.py
graphrouter/
  policy.py
  artifact_loader.py
avengers_pro/
  policy.py
  artifact_loader.py
brainbase_custom/
  policy.py
  artifact_loader.py

registry/
  model_registry.yaml
  policy_registry.yaml
  tenant_policy_config.yaml
  capability_schema.yaml

executor/
  portkey_executor.py
  workflow_executor.py
  response_normalizer.py

observability/
  kafka_events.py
  postgres_writer.py
  trace_schema.py

evals/
  replay_harness.py
  benchmark_runner.py
  policy_comparator.py
  public_benchmarks/
  brainbase_internal_evals/

training/
  dataset_builder.py
  reward_builder.py
  offline_train.py
  online_update.py
  distillation_jobs.py

admin/
  model_lifecycle.py
  policy_lifecycle.py
```

```
rollout_manager.py  
rollback_manager.py
```

Part 11: Final Product Definition

What Brainbase Router is

Brainbase Router is a production router runtime that exposes an OpenAI-compatible model endpoint while internally selecting the best model, model sequence, cascade, or workflow for each request.

What makes it valuable

```
modular router brain  
Portkey execution gateway  
broad model marketplace  
customer-specific routing  
agent-step routing  
cost/latency optimization  
training data loop  
versioned router artifacts  
eval-driven promotion
```

What should remain stable

```
customer API  
Portkey execution integration  
logging  
billing  
auth  
Kafka/Postgres/S3 pipeline  
eval harness  
training pipeline  
admin dashboard
```

What should be replaceable

```
router algorithm  
router artifact  
model registry  
model capability profiles
```

```
tenant routing config  
fine-tuned LLMs
```

Final architecture principle

```
Brainbase should not build a HyDRA wrapper.  
Brainbase should build a router runtime where HyDRA is one installable brain.
```

Final product statement

Brainbase Router is a modular, trainable, OpenAI-compatible routing product. Customers call it like a normal model. Internally, Brainbase uses a replaceable RouterPolicy protocol to choose the best model, cascade, workflow, or agent-step execution plan. Portkey executes the selected plan, while Kafka/Postgres capture outcomes for evaluation and future training. New router algorithms, fine-tuned routers, and new foundation models can be added as versioned artifacts or registry entries without rewriting the product.

Final simplest version

```
Customer calls:  
  brainbase-router-v1  
  
Brainbase runtime loads:  
  active RouterPolicy  
  
RouterPolicy returns:  
  RoutePlan  
  
Portkey executes:  
  selected model or plan  
  
Brainbase logs:  
  cost, latency, quality, success  
  
Brainbase improves:  
  train new router, shadow test, promote
```

That is the product.